Final Technical Approach - Tech Team Build Guide

Multi-Source Graph-Based Shell Company & Financial Risk Detection (MCA + CERSAI + RBI Wilful Defaulters)

This document explains the final approach end-to-end: what data you have, how the separate CSV files connect to each other, which graph algorithm we use for clustering and why, and the exact build sequence to follow.

1. What We Are Building - One-Paragraph Summary

We convert three separate data sources - company registration data (MCA-style), loan/security-interest data (CERSAI-style), and wilful-defaulter classification data (RBI-style) - into a single connected graph. Companies, directors, addresses, and lenders become nodes; relationships between them become edges. We then run a community-detection algorithm (Louvain) on this graph to find tightly-connected clusters of companies. Clusters that show multiple suspicious signals at once (shared directors, shared address, shared lender, already-flagged defaulters, incorporation happening in a short burst) get ranked highest on an investigator watchlist. The system does not claim to prove fraud - it surfaces structurally unusual company networks for human review.

2. The Dataset - Five CSV Files, Not One Master File

This is intentional, not a limitation. Real government data lives in separate systems (MCA, CERSAI, RBI are different regulatory bodies with different databases) and every company does not appear in every source (most companies have no loan record; very few are defaulters). We keep them as five separate tables, linked by a shared key (CIN), exactly like a relational database. Do NOT try to flatten these into one file before processing - join them in code instead (Section 4 shows exactly how).

File	What it contains	Join key	Rows
mca_companies.csv	One row per company: CIN, name, registered address, city/state, incorporation date, authorized capital, paid-up capital, status	CIN (primary key)	467
mca_directors.csv	One row per director-company link: CIN, DIN, director name. A company can have 1-2 rows here (1-2 directors).	CIN (foreign key)	608
cersai_security_interests.csv	One row per loan/security-interest record: CIN, borrower name, lender name, security type, asset description, charge amount, charge date. Not every company has a row here.	CIN (foreign key)	116
rbi_wilful_defaulters.csv	One row per wilful-defaulter classification: CIN, company name, lender name, default amount, classification date, reason. Very few companies have a row here - this should stay rare.	CIN (foreign key)	14
ground_truth.csv	CIN + label (normal / fraud_ring_A / fraud_ring_B / fraud_ring_C / legit_edge_case). For evaluation only. Never feed this into the detection algorithm.	CIN	467

Why row counts differ across files

467 companies exist, but only 116 have a loan record and only 14 are flagged defaulters. This is realistic and correct - in the real world, most companies never appear in CERSAI or the wilful defaulter list. If your merged data shows every company having a loan and being a defaulter, something is wrong with how the files were joined.

3. What The Dataset Actually Contains

The 467 companies are not random noise - they are deliberately structured so the detection system has something real to find, and something real to correctly ignore.

Group	Size	Design purpose
Background (normal) companies	440	Realistic, independent companies - spread-out addresses, mostly 1-2 directors, about 18% have a genuine unrelated loan. This is what a normal graph region should look like.
Legitimate high-degree edge case	7	All share one address (like a registered-agent office) but each has a different director. Tests that the system does not falsely flag shared-address-alone as fraud.
Fraud Ring A - classic dense	9	Kolkata. 3 shared directors, 1 shared address, 1 shared lender, 6 of 9 already flagged as wilful defaulter. The clearest, easiest-to-detect case.
Fraud Ring B - subtler	6	Pune. Only 2 shared directors, 2 different address variants, only 2 of 6 flagged defaulter. Tests whether weaker signal combinations still get caught.
Fraud Ring C - defaulter-heavy	5	Ahmedabad. Tight 6-day incorporation burst, all 5 already flagged defaulter. Tests a different signal combination.

Three distinct networks, not one, means the final ranked watchlist should show multiple genuinely different suspicious clusters - which is what makes the investigator dashboard demo convincing, rather than looking like a single hardcoded example.

4. How To Actually Use These Files In Code

4.1 Load all four real tables (skip ground_truth for the detection pipeline itself)

```
import pandas as pd

companies = pd.read_csv("mca_companies.csv")
directors = pd.read_csv("mca_directors.csv")
cersai    = pd.read_csv("cersai_security_interests.csv")
wilful    = pd.read_csv("rbi_wilful_defaulters.csv")

print(companies.shape)   # (467, 9)
print(directors.shape)   # (608, 3)
print(cersai.shape)      # (116, 7)
print(wilful.shape)      # (14, 6)
```

4.2 Build ONE unified graph from all four tables

This is the key step. Every table contributes nodes and/or edges to the same graph object - this is what makes it a multi-source system rather than four separate analyses.

```
import networkx as nx

G = nx.Graph()

# Layer 1: companies + their registered addresses (from mca_companies.csv)
for _, row in companies.iterrows():
    G.add_node(row["CIN"], type="company", name=row["COMPANY_NAME"])
    G.add_node(row["REGISTERED_OFFICE_ADDRESS"], type="address")
    G.add_edge(row["CIN"], row["REGISTERED_OFFICE_ADDRESS"])

# Layer 2: directors (from mca_directors.csv)
for _, row in directors.iterrows():
    G.add_node(row["DIN"], type="director", name=row["DIRECTOR_NAME"])
    G.add_edge(row["DIN"], row["CIN"])

# Layer 3: lenders (from cersai_security_interests.csv)
for _, row in cersai.iterrows():
    G.add_node(row["LENDER_NAME"], type="lender")
    G.add_edge(row["CIN"], row["LENDER_NAME"])

# Layer 4: wilful-defaulter flag (from rbi_wilful_defaulters.csv)
# This is a NODE ATTRIBUTE on the existing company node, not a new node/edge
defaulter_cins = set(wilful["CIN"])
for cin in defaulter_cins:
    if G.has_node(cin):
        G.nodes[cin]["wilful_defaulter_flag"] = True

print(f"Total nodes: {G.number_of_nodes()}, Total edges: {G.number_of_edges()}")
```

After this step, companies, directors, addresses, and lenders are all connected nodes in one graph. This single merged graph is what Louvain community detection runs on next - the multi-source signal comes from the graph structure itself, not from running three separate algorithms and combining results.

5. The Clustering Algorithm - Louvain Community Detection

5.1 What it does, in plain terms

Louvain finds groups of nodes that are much more densely connected to each other than to the rest of the graph. In our graph, a fraud ring of companies sharing directors, an address, and a lender forms exactly this kind of dense pocket - Louvain's job is to find that pocket automatically, without being told in advance where to look.

5.2 Why Louvain specifically (not another algorithm)

Reason	Detail
Built into NetworkX	networkx.algorithms.community.louvain_communities - zero extra dependencies to install.
Fast on sparse graphs	Near-linear time in practice. Runs live during a demo without lag, even at the roughly 1,200-1,600 total-node graph scale here.
Designed for exactly this shape of problem	Finding dense sub-groups in a mostly-sparse background graph is precisely what Louvain optimizes for (maximizing modularity - how much more connected a group is internally than random chance would predict).
Backed by the fraud-detection literature already referenced	Recent papers (e.g. Informatica 2025 on Louvain-based fraud detection in transaction networks) use Louvain for structurally identical problems - this is not an arbitrary choice.

5.3 The alternative considered and ruled out

Girvan-Newman (edge-betweenness based community detection) is more precise on small graphs but is computationally expensive and does not scale - wrong choice once the graph grows beyond a few thousand nodes. Mention this only if asked whether alternatives were considered.

5.4 Running it

```
from networkx.algorithms.community import louvain_communities

communities = louvain_communities(G, seed=42)   # fixed seed = reproducible demo results
print(f"Found {len(communities)} communities")
```

6. Scoring Each Cluster - The Five Signals

Louvain tells us which companies are grouped together. It does not tell us which groups are suspicious. The composite score below combines signals from all three data sources into one ranked risk score per cluster.

Signal	Source file	What it measures
Director degree	mca_directors.csv	Average number of companies linked to each director in the cluster
Address degree	mca_companies.csv	Average number of companies sharing each address in the cluster
Incorporation burst	mca_companies.csv	How tightly clustered the incorporation dates are within the cluster
Shared-lender density	cersai_security_interests.csv	Whether cluster members borrow from the same lender rather than a spread of different banks
Wilful-defaulter ratio	rbi_wilful_defaulters.csv	Percentage of the cluster already flagged as a wilful defaulter

6.1 Composite score formula

```
score = (0.25 * avg_director_degree
       + 0.25 * avg_address_degree
       + 0.15 * incorporation_burst_score
       + 0.15 * shared_lender_density
       + 0.20 * wilful_defaulter_ratio)
```

Document these weights and the reasoning (director + address weighted highest as the most direct structural fraud signals; wilful-defaulter ratio weighted meaningfully as the strongest real-world confirming signal, but not so high it overrides everything else - Ring B must still be catchable even with a low defaulter ratio).

6.2 Critical rule: set thresholds from background data before testing

Compute the mean and standard deviation of director-degree and address-degree using only the 440 background companies (label equals normal in ground_truth.csv, used only to select this subset for threshold-setting, never as a model input). Fix the threshold. Only then run detection against the full dataset including all three rings. Reversing this order - tuning thresholds after seeing whether the rings get caught - would invalidate the experiment and will not survive judge questioning.

7. Evaluating The Result (Not "Train/Test")

This is a rule-based, unsupervised system. No model is being trained, so a traditional train/test split does not apply. Validation instead works like this:

1. Run Louvain + scoring on the full merged graph (all 467 companies, all three rings included).

2. Rank all detected clusters by composite score.

3. Using ground_truth.csv only for this check (never fed into the algorithm itself), confirm where each of Ring A, Ring B, and Ring C land in the ranked output.

4. Confirm the legitimate edge case (7 companies, shared address only) scores meaningfully lower than the three rings, despite also having above-average address-degree.

5. Report: detection rank per ring, false-positive rate on the 440 background companies, and this qualitative distinction on the edge case.

```
for rank, community in enumerate(ranked_clusters):
    cins_in_cluster = [n for n in community if G.nodes[n].get("type") == "company"]
    labels = ground_truth_df[ground_truth_df["CIN"].isin(cins_in_cluster)]["label"].unique()
    if any(l.startswith("fraud_ring") for l in labels):
        print(f"Rank {rank+1}: {labels} detected, score={community_score:.1f}")
```

8. Tech Stack - Minimum, No New Technology

Tool	Purpose	Install
Python 3	Core language	-
pandas	Load and join the CSV files	pip install pandas
networkx	Graph construction + Louvain community detection (built in)	pip install networkx
pyvis	Interactive force-directed graph visualization for the demo	pip install pyvis
numpy	Mean/stdev calculations for thresholding	pip install numpy

That is the entire dependency list. No Neo4j, no Docker, no separate backend service, no ML library - all deliberately excluded to keep the build achievable in the time available.

9. Build Sequence - Follow In This Order

1. Load all four CSVs with pandas, confirm row counts match what is documented in Section 2.

2. Build the unified graph (Section 4.2) - verify node/edge counts look reasonable (roughly 467 company nodes plus around 460 unique addresses plus around 610 director nodes plus a handful of lender nodes).

3. Compute director-degree and address-degree across the full graph, but calculate mean/stdev thresholds using only the background (normal) companies.

4. Run louvain_communities(G, seed=42) and print how many communities were found.

5. Score every community using the five-signal composite formula (Section 6.1).

6. Rank communities by score and check where Ring A, Ring B, Ring C, and the legitimate edge case land, using ground_truth.csv for this check only.

7. Build the pyvis visualization: colour nodes by type (company/director/address/lender), and colour the top-ranked clusters distinctly so they visually stand out against the background.

8. Pre-render the final graph HTML before the demo rather than generating it live, to avoid any loading delay in front of judges.

This document reflects the final build approach. Data is synthetic (with realistic Indian names and correctly-formatted CIN/DIN identifiers), grounded in real regulatory data sources (MCA21, CERSAI, RBI Wilful Defaulter framework) and a real reference case (the Rs 734 crore GST ITC fraud, 135 shell companies, ED prosecution July 2025) that shaped the signal design.